



Métodos de Pesquisa e Ordenação em Estruturas de Dados

UNIDADE 06

Estrutura de dados: Ordenando coisas

Olá! O nome desta disciplina é um pouco longo, não é? Quer dizer: Métodos de Pesquisa e Ordenação de Estruturas de Dados implica que precisamos conhecer diferentes estruturas de dados (o que já fizemos), mas também, precisaremos entender diferentes métodos de pesquisa e métodos de ordenação. Bom, este é o momento para isso.

No início desta unidade, embarcaremos em uma jornada pelo mundo dos algoritmos de ordenação, um componente fundamental da ciência da computação e uma ferramenta essencial no arsenal de qualquer desenvolvedor competente. A ordenação é mais do que somente arrumar elementos em uma sequência específica; é um pilar-chave na organização de dados, essencial para a eficiência e eficácia de algoritmos de busca, estruturas de dados e sistemas de gerenciamento de banco de dados.

Por isso, nesta unidade, analisaremos as mecânicas de vários algoritmos de ordenação, cada um com suas características únicas, adaptados para atender a diferentes necessidades e cenários. Desde métodos simples e intuitivos, como o *bubble sort*, até técnicas mais sofisticadas e eficientes, como o *quick sort* e o *merge sort*, cada algoritmo é uma peça crucial no quebra-cabeça da computação.

À medida que avançamos, abordamos não apenas os princípios básicos e a lógica por trás de cada método de ordenação, mas também suas implicações práticas, eficiência, e casos de uso ideais. Essa compreensão nos permite apreciar a beleza e a elegância desses algoritmos, assim como tomar decisões informadas sobre qual algoritmo escolher em uma determinada situação. Seja lidando com pequenos conjuntos de dados em aplicações do dia a dia ou gerenciando grandes volumes de informação em sistemas complexos, os algoritmos de ordenação desempenham um papel vital em tornar a informação acessível e rápida de ser organizada.

Vamos lá?

| ORDENANDO DADOS

Agora, por que precisamos nos preocupar em ordenar dados? Vamos supor que estamos criando uma lista em Python. Para ordená-la bastaria chamar algo como *lista.sort()*. Fácil, né? Então, por que isso é relevante para nós?

Veja: estes métodos de ordenação de dados são fundamentais na ciência da computação e na prática de programação por vários motivos. O primeiro é **emprego**: em empresas maiores de TI é comum perguntarem aos candidatos se conhecem e sabem diferenciar algumas técnicas de ordenação de dados.

O segundo motivo é a **versatilidade**: diferentes algoritmos de ordenação têm diferentes forças e fraquezas. Alguns são rápidos para pequenos conjuntos de dados, já outros são mais eficientes para dados já parcialmente ordenados, e alguns são mais adequados para grandes conjuntos de dados (*big data*). Por isso, conhecer uma variedade de métodos permite que você possa escolher o mais eficiente para cada situação específica.

O terceiro motivo é a **complexidade computacional**: cada método de ordenação tem uma complexidade computacional que descreve o quão eficiente ele é em relação ao número de operações necessárias. Às vezes, um método que funciona bem nos seus testes pode se tornar bem lento no dia a dia por não ser escalável para grandes volumes de dados.

Atrelado a isso também temos desafios relacionados à **memória**: você pode até escolher um método que seja mais rápido, mas isso pode acabar consumindo muita memória do seu dispositivo e, na prática, pode acabar travando o seu celular ou o seu computador. Por isso, você pode acabar preferindo um algoritmo mais leve e que não necessariamente possa ser mais rápido.

Também existe um motivo relacionado a **aprender algoritmos**: entender melhor estas estratégias te ajudam a criar algoritmos que sejam mais eficientes de forma crítica. Afinal, conhecer novas linguagens de programação é apenas uma pequena parte da tarefa: você também precisa saber **como criar bons algoritmos** também.

Bom, acho que você entendeu o porquê de aprendermos isso, certo? Agora, do ponto de vista de algoritmos, em quais casos precisamos ordenar dados? Veja só: em muitos sistemas, como filas de atendimento ou gerenciamento de tarefas, a ordenação ajuda a determinar **prioridades**. Por exemplo, em um sistema operacional, processos podem ser ordenados por sua prioridade de execução para que o gerenciador de tarefas saiba qual processo executar a seguir. Além disso, dados ordenados são mais fáceis de **comparar e analisar**. Imagine que você tem uma lista de pontuações de estudantes em uma prova e quer encontrar a mediana. Se a lista estiver desordenada, você terá que passar por todos os dados para encontrar a mediana. Se estiver ordenada, você pode simplesmente pegar o valor do meio dessa lista.

Também acaba sendo muito mais fácil de **encontrar e remover valores duplicados** se as informações estão ordenadas, já que os valores duplicados estarão juntos. Da mesma forma, é muito mais **rápido** de encontrarmos informações quando as coisas estão organizadas (ou você conseguiria encontrar o que quer em um lugar todo bagunçado?).



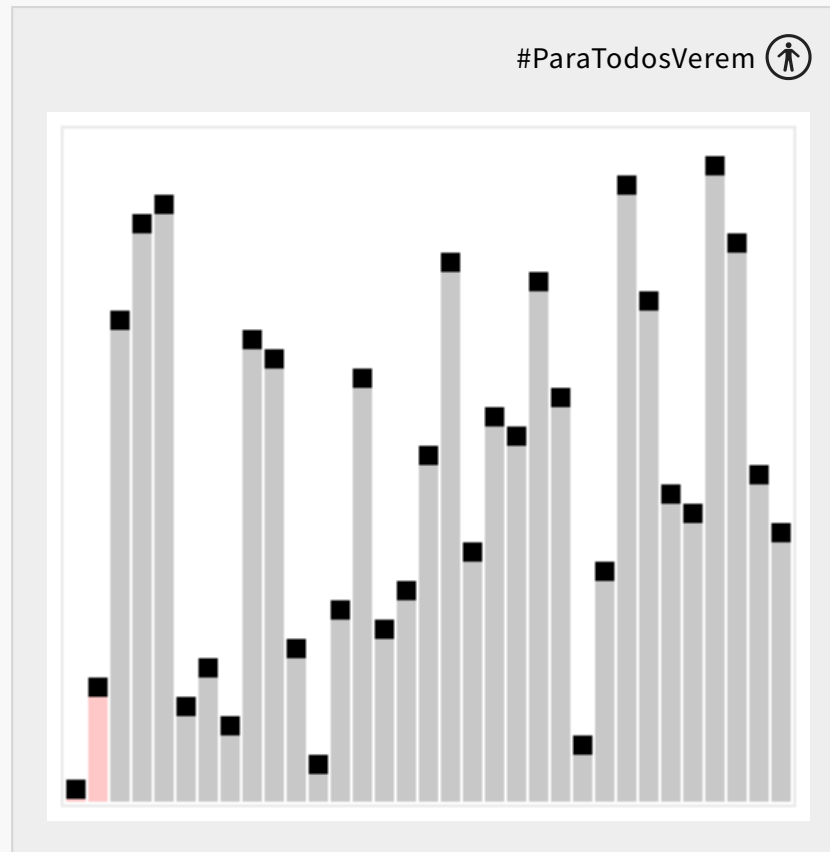
Técnicas de ordenação

Acredito que podemos dividir os métodos de ordenação entre alguns tipos: temos aqueles métodos que são mais simples (mas não necessariamente mais eficientes); aqueles que são eficientes (mas não necessariamente muito fáceis); e aqueles que são lineares (que não comparam os valores para que possam ordenar).

Vamos começar pelos mais fáceis, beleza?

Métodos simples de ordenação

Vamos começar com um exemplo e, para isso, precisaremos padronizar alguns entendimentos. Observe a animação a seguir. Aqui, considere que cada barra representa um valor: quanto menor a barra, menor o número. Imagine também que este número pode representar conceitos como idade, salário, altura, ou qualquer outro valor numérico.



Fonte:
https://commons.wikimedia.org/wiki/File:Sorting_selection_sort_anim.gif

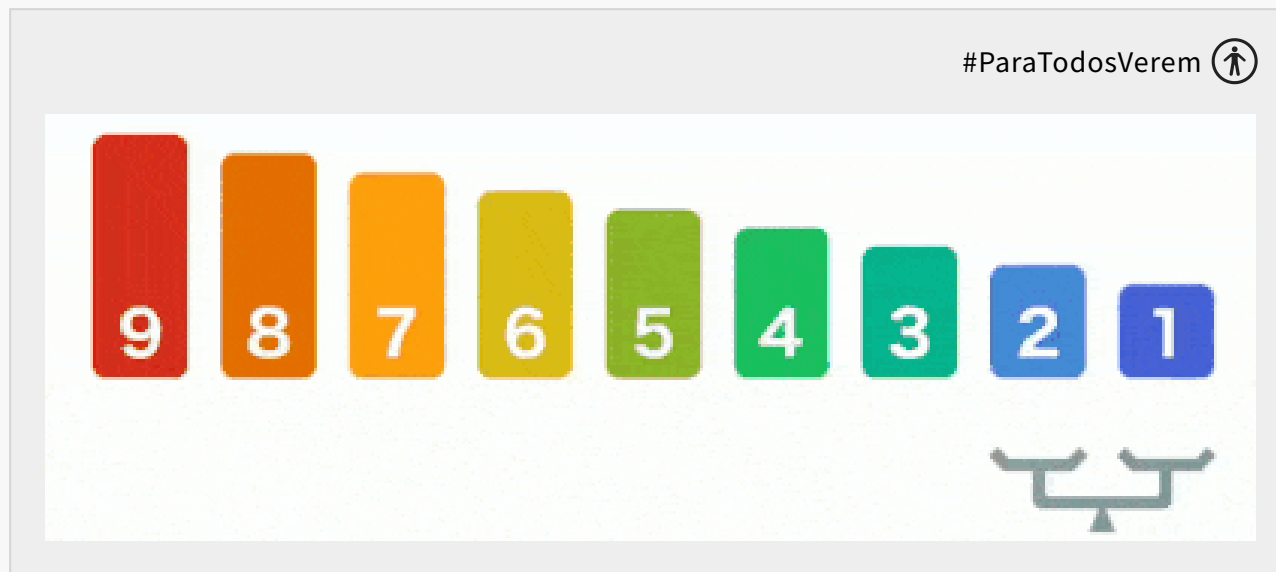
Logo, o nosso grande desafio é o de ordenar essas barras: queremos, no final das contas, algo parecido como uma escada. O menor valor é acompanhado pelo segundo menor valor. Por sua vez, este segundo maior valor é acompanhado pelo terceiro menor valor, e assim sucessivamente.

Também veja que às vezes selecionamos um valor (fundo em vermelho claro) e trocamos de lugar com outro valor (fundo em vermelho escuro). Repetimos este passo a passo (ou algoritmo) algumas vezes. Pegou a ideia? Bom, vamos para alguns métodos a partir de agora. Os métodos mais simples que gostaria de mostrar a você são: ***bubble sort***, ***insertion sort*** e ***selection sort***. Aqui, “sort” sempre quer dizer “ordenação”.

→ **BUBBLE SORT: FÁCIL DE IMPLEMENTAR, MAS INEFICIENTE**

Pense em uma fila de crianças, e cada uma possui um tamanho diferente. Por enquanto a nossa fila está desordenada: temos primeiramente uma criança bem alta, depois uma mais baixa, depois uma mais baixa ainda, depois outra alta, e assim por diante. O nosso desafio é que as crianças se organizem em uma fila.

Agora, vamos dizer que cada uma das crianças comparam o seu próprio tamanho com as crianças que estão ao seu lado, e trocam de lugar se elas perceberem que estão na ordem errada. Elas farão isso várias vezes até que a fila esteja totalmente ordenada. Para ficar mais fácil, vejamos um exemplo mais visual:



Fonte: Moriteru Ishida/App: Algorithms: Explained and Animated.

Agora, de forma mais formal: o ***bubble sort*** é um algoritmo simples de ordenação que percorre repetidamente uma lista, compara elementos adjacentes e os troca de lugar se estiverem na ordem errada. Este processo é repetido até que a lista esteja ordenada. Como você deve imaginar, a **eficiência do bubble sort não é das melhores, principalmente em listas grandes.**

Cada passagem através da lista cria uma bolha de comparações e trocas: daí o nome de “*bubble sort*”. Para entender isso, pense em um tanque de água com várias bolas de tênis flutuando na superfície. Se você quiser ordená-las da mais suja para a mais limpa, você poderia comparar cada par de bolas próximas, empurrando a mais suja para um lado do tanque. Eventualmente, a bola mais suja chega ao final do tanque, e você repete o processo até que todas estejam ordenadas.

Apesar de não ser eficiente, o *bubble sort* é intuitivo e fácil de implementar, o que o torna um bom algoritmo de ordenação para entender os conceitos básicos de como os algoritmos de ordenação funcionam. No entanto, na prática, ele é usado principalmente em contextos educacionais ou em listas que são quase ordenadas, em que seu desempenho pode ser surpreendentemente bom. Se você estivesse organizando uma pequena estante de livros por cor e todos eles já estivessem **quase** na ordem correta, ajustar cada par que está fora de ordem seria um processo rápido, semelhante ao que acontece no melhor caso do *bubble sort*. Agora, em aplicações de big data esta técnica realmente não é recomendada.

→ **INSERTION SORT: SIMPLES, E BOM PARA BASES PEQUENAS**

Para entendermos este algoritmo pensemos em outro exemplo. Imagine que você tem uma série de livros espalhados sobre uma mesa e deseja organizá-los por ordem de publicação. Você começa com um único livro, considerando-o como uma sequência ordenada. Em seguida, você pega outro livro e coloca-o antes ou depois do primeiro livro, dependendo do ano de publicação. Agora, com dois livros ordenados, você pega o próximo livro e o insere na posição correta entre eles, continuando esse processo até que todos os livros estejam na sequência ordenada na mesa. Para ficar mais claro, veja o exemplo a seguir:



Fonte: Moriteru Ishida/App: Algorithms: Explained and Animated.

Então, o *insertion sort* é um outro tipo algoritmo de ordenação que constrói a sequência final (ordenada) **um elemento de cada vez**. Você pode comparar este método ao jeito em que as pessoas ordenam cartas em suas mãos durante um jogo de baralho: começando com uma carta, você pega uma nova carta por vez e a coloca na posição correta entre as já existentes, de forma que a sequência parcial sempre se mantém ordenada.

Agora, quando usamos este método? O *insertion sort* é eficiente para conjuntos de dados pequenos ou quando a lista está quase ordenada, em que poucas movimentações são necessárias. Por outro lado, ela sofre com um problema parecido com o *bubble sort*: ele se torna menos eficiente à medida que o tamanho da lista aumenta por causa da quantidade de comparações e movimentações necessárias.

→ SELECTION SORT: RÁPIDO E ELEGANTE

Lembra do exemplo da fila de crianças com diferentes alturas do *bubble sort*? Então: no *selection sort*, o processo seria um pouco diferente. Primeiramente: procuraríamos a criança mais baixa de todas. Depois de encontrá-la, você a convidaria para se posicionar no início da fila. Em seguida, você repetiria esse processo para a próxima posição na fila, percorrendo todas as crianças restantes para encontrar a próxima mais

baixa e assim sucessivamente, até que todas estivessem ordenadas. Veja o exemplo a seguir para ficar mais claro:



Fonte: Moriteru Ishida/App: Algorithms: Explained and Animated.

Veja que o selection sort é um algoritmo de ordenação que divide a lista em **duas partes: uma com itens já ordenados e outra com os itens que ainda precisam ser ordenados**. O processo consiste em encontrar o menor (ou maior, dependendo da ordem desejada) elemento da parte não ordenada e trocá-lo com o primeiro elemento não ordenado.

Este método geralmente faz menos trocas de posição entre os valores para ordenar a lista. No entanto, mesmo que a lista já esteja ordenada ou quase ordenada, ele ainda percorre toda a lista para confirmar que cada elemento é o menor, o que o torna **ineficiente para estruturas de dados grandes**. É um algoritmo simples e didático, usado frequentemente para fins educacionais para ilustrar o conceito de ordenação, mas não é eficaz para ordenação em larga escala. É por isso que em seguida veremos algoritmos mais avançados como o *quick sort*, o *merge sort*, ou o *heap sort*.

Agora, quando usamos estes métodos na realidade? Vamos por partes: para começo de conversa, o ***bubble sort*** raramente é usado em sistemas nas empresas por ser ineficiente com conjuntos de dados maiores. No entanto, existem alguns casos de uso em que aplicamos este método:

+ Cursos de TI

É frequentemente utilizado como uma ferramenta de ensino para introduzir conceitos de algoritmos de ordenação devido à sua simplicidade.

+ Dispositivos com pouquíssimos recursos computacionais

Nestes casos, a simplicidade do código é mais importante do que a eficiência da execução.

+ Listas quase ordenadas

Se a lista estiver quase ordenada, o *bubble sort* pode ser eficiente. Afinal, ele precisa de apenas uma passagem pela lista sem trocas para verificar se a lista está ordenada.

Já o *insertion sort* tem várias aplicações práticas, especialmente em que a inserção é frequente e o conjunto de dados é pequeno:

+ Ordenação em tempo real

É útil em situações em que os dados chegam em tempo real e a lista deve ser mantida ordenada a todo momento, como a organização da fila de impressão em um servidor de impressão ou de uma impressora.

+ Pequenas listas

Em listas pequenas, como poucos registros em um banco de dados ou pequenas listas de contatos em um telefone, o *insertion sort* pode ser mais rápido devido à sua simplicidade e por ser mais leve de ser executado.

+ Listas quase ordenadas

Similar ao *bubble sort*, se uma lista está quase ordenada, o *insertion sort* pode ser muito eficiente.

Finalmente, o *selection sort* também não é muito usado em aplicações práticas devido à sua ineficiência, mas tem casos de uso específicos:

+ Cursos de TI

Assim como o *bubble sort*, é útil em situações educacionais para explicar os fundamentos dos algoritmos de ordenação.

+ Sistemas simples

É comum usarmos esta técnica em sistemas embarcados ou de hardware com limitações de memória, em que a complexidade do código deve ser minimizada.

+ Situações com trocas caras

Também podemos preferir esta técnica em sistemas em que o custo de troca é significativamente maior do que o custo de comparação. Exemplos disso são sistemas com armazenamento lento (como HDs em vez de SSDs) ou bancos de dados altamente normalizados.

Em aplicações do mundo real, especialmente aquelas que lidam com grandes conjuntos de dados, algoritmos de ordenação mais eficientes como *quick sort*, *merge sort* ou algoritmos baseados em *heap* são geralmente preferidos. Dito isso, é importante que você conheça estes algoritmos mais simples porque existe uma chance relativamente alta de que algum entrevistador queira que você responda sobre estes algoritmos.

Métodos eficientes de ordenação

Também temos algoritmos que são mais complexos de se implementar. Por outro lado, ganhamos mais eficiência ao usá-los. Estes algoritmos são mais comuns de serem usados dentro das empresas e pelas linguagens de programação.

→ MERGE SORT: DIVIDINDO PARA CONQUISTAR

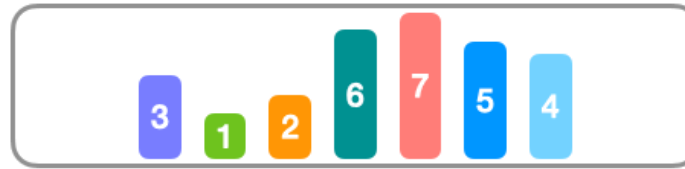
O *merge sort* é um algoritmo de ordenação **eficiente, baseado na abordagem *divide-and-conquer*** (dividir para conquistar). Ele divide o conjunto de dados em partes cada vez menores, resolve cada uma delas e depois as combina (ou "*merge*") para obter a solução final ordenada.

Vamos usar um exemplo da vida real para entender melhor como essa estratégia funciona, beleza? Bom, imagine que você tem uma playlist enorme com centenas de músicas que deseja organizar por gênero musical. O problema dessa tarefa é que você precisará fazer isso manualmente. Tentar ordenar toda a playlist de uma vez pode ser confuso e demorado. Então, você decide usar a abordagem do *merge sort*.

Primeiramente, você divide a *playlist* em duas metades, depois essas metades em quartos, e assim por diante, continuando a dividir até que cada "*subplaylist*" tenha apenas uma música. Essa etapa de dividir a *playlist* em partes cada vez menores é como o primeiro passo do *merge sort*, em que o conjunto de dados é dividido repetidamente pela metade.

Agora, você começa a combinar essas *subplaylists*. Você pega duas *subplaylists* com uma música cada e as combina em uma *subplaylist* de duas músicas, organizando-as por gênero. Em seguida, você pega outra dupla de *subplaylists* e faz o mesmo, e continua esse processo. Cada vez que você combina duas *subplaylists*, você as ordena. Isso é semelhante à fase de mesclagem do *merge sort*, em que duas listas menores são combinadas em uma lista maior e ordenada.

Nessa estratégia, você continua combinando e ordenando *subplaylists*, tornando-as cada vez maiores, até que todas as músicas estejam de volta em uma única *playlist*, agora completamente organizada por gênero. Claro: eu sei que essa estratégia é mais complexa de explicar e de implementar em código, mas acaba sendo bem mais eficiente conforme temos mais dados. Veja a imagem a seguir para um exemplo do *merge sort*. Veja que na parte superior temos uma lista desordenada.



Fonte: Moriteru Ishida/App: Algorithms: Explained and Animated.

Cada linha neste exemplo representa uma divisão dos dados. Observe que dividimos em duas metades (segunda linha), depois cada uma dessas metades em mais duas metades cada, totalizando quatro partes (terceira linha), e chegamos a ter somente um item separado (quarta linha).

Ao terminarmos, começamos a juntar e ordenar cada uma dessas partes até termos a nossa lista ordenada. Em resumo:

1. **Dividir:** inicialmente, o conjunto de dados é dividido em duas metades. Se o conjunto já tiver apenas um elemento, ele é considerado ordenado. Se tiver mais de um elemento, continuamos dividindo cada metade em metades menores, recursivamente, até que cada subdivisão tenha apenas um elemento.
2. **Conquistar (ordenar e mesclar):** depois que o conjunto de dados é dividido em unidades com um único elemento, começamos a combinar essas unidades. Durante a combinação (ou mesclagem), ordenamos os elementos. Duas unidades de um elemento são

combinadas para formar uma unidade de dois elementos ordenados, duas unidades de dois elementos são combinadas para formar uma unidade de quatro elementos ordenados, e assim por diante.

3. **Mesclar:** continuamos esse processo de mesclar e ordenar até que todas as unidades pequenas sejam combinadas de volta em um único conjunto ordenado.

O *merge sort* é particularmente eficaz porque é estável (não altera a ordem de elementos iguais), funciona bem para listas grandes e sua eficiência é garantida, sempre executando em tempo $O(n \log n)$, em que n é o número de itens a serem ordenados. Isso o torna uma escolha confiável para sistemas de ordenação em grande escala, como bancos de dados e aplicações que requerem alta confiabilidade e previsibilidade no desempenho de ordenação.

→ QUICK SORT: USANDO PIVÔS

O *quick sort* é outro tipo de algoritmo de ordenação **eficiente e baseado no *divide-and-conquer***. A diferença aqui é a que temos o conceito de **pivô**: um elemento escolhido aleatoriamente para usarmos como base para a ordenação de outros elementos. Veja um resumo de como ele funciona:

1. **Escolha do Pivô:** primeiramente, um elemento é escolhido da lista para ser o pivô. A escolha do pivô pode ser feita de várias maneiras: pode ser o primeiro elemento, o último, o elemento do meio, ou até mesmo um elemento escolhido aleatoriamente.
2. **Particionamento:** o próximo passo é reorganizar a lista de tal forma que todos os **elementos menores do que o pivô fiquem à sua esquerda e todos os elementos maiores fiquem à sua direita**. Esta operação é conhecida como particionamento. Após o particionamento, o pivô está em sua posição final ordenada.
3. **Recursão:** O processo é então aplicado recursivamente às duas sublistas, a dos elementos à esquerda do pivô e a dos elementos à direita.
 - a. Isso significa que repetiremos o processo de escolha de um pivô e particionar as sublistas, e então aplicar o *quick sort* nas sublistas resultantes.
4. **Conclusão:** Este processo de divisão e conquista continua até que as sublistas sejam suficientemente pequenas (geralmente quando têm apenas um elemento). Este será o momento em que a lista inteira estará ordenada.

Veja a imagem a seguir para um exemplo de particionamento e recursão. Nela, temos sempre um pivô (P), o qual inicialmente foi o último elemento da lista. Também temos as sublistas selecionadas a cada passo (a linha em cinza, embaixo da animação), e a posição à esquerda (L) e direita (R) para reposicionar o pivô.



Fonte: Moriteru Ishida/App: Algorithms: Explained and Animated.

Para ajudar, pensemos também em uma analogia usando o passo a passo apresentado. Imagine que você é o bibliotecário responsável por organizar todos os livros de uma grande biblioteca por ano de publicação. Para usar o *quick sort* nesse cenário, você faria o seguinte:

1. **Escolha do pivô:** primeiramente, você seleciona um livro aleatoriamente como seu ponto de referência ou "pivô". Vamos dizer que você escolhe um livro aleatório publicado em 2009.
2. **Particionamento:** em seguida, você reorganiza a biblioteca de modo que todos os livros publicados antes de 2009 fiquem de um lado do pivô, e todos os livros publicados após 2009, do outro lado. Agora, o livro de 2009 está na posição correta, na ordem de anos de publicação.

3. **Recursão:** agora você aplica o mesmo processo nas duas seções da biblioteca que foram criadas – uma com livros publicados antes de 2009 e outra com livros publicados depois. Em cada seção, você escolhe um novo livro como pivô e reorganiza os livros da mesma maneira.

a. Por exemplo: agora você pode fazer a mesma coisa escolhendo como pivô um livro de 2003 e, no outro lado, um livro de 2015.

Depois disso, você repetirá o processo com outros anos.

4. **Conclusão:** você continua esse processo de escolher um pivô, particionar, e depois aplicar o *quick sort* nas seções menores da biblioteca, até que cada seção tenha um ou nenhum livro. Ao final, todos os livros estarão organizados por ano de publicação.

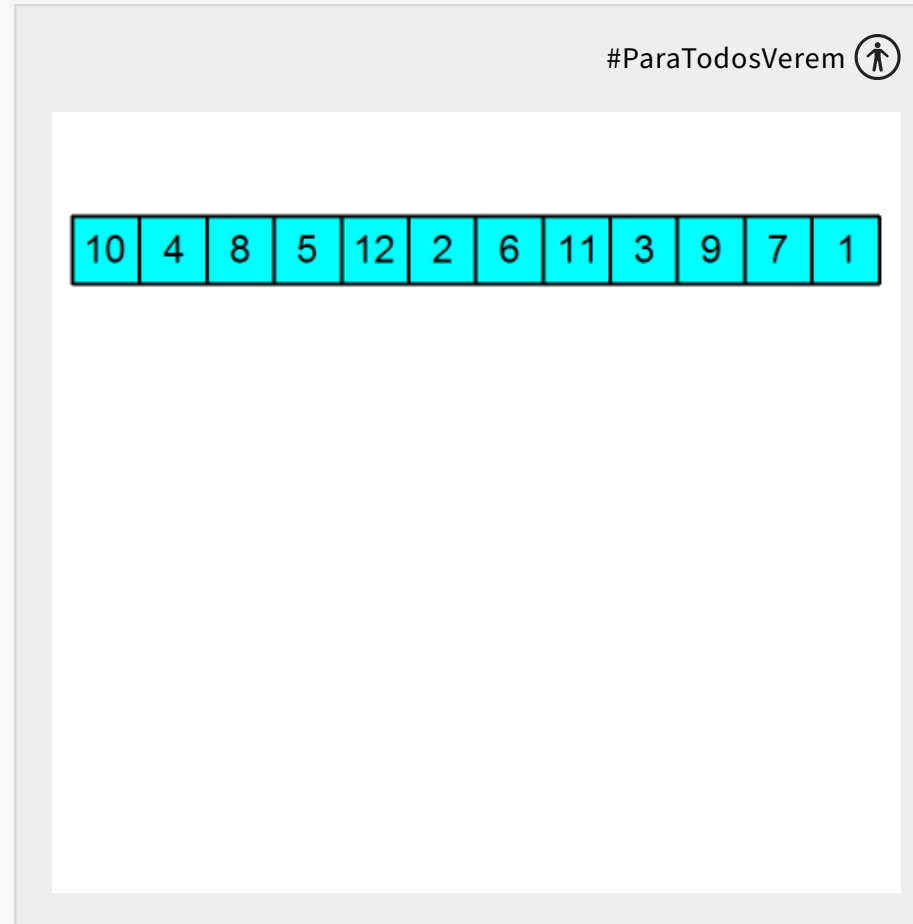
O *quick sort* é particularmente eficiente para grandes listas, e sua eficiência média é geralmente melhor do que outros algoritmos de ordenação simples, como o *bubble sort*, o *insertion sort* e o *selection sort*. No entanto, sua performance pode variar dependendo de como o pivô é escolhido. Na pior das hipóteses, especialmente quando o menor ou maior elemento é consistentemente escolhido como o pivô, a eficiência do *quick sort* pode se degradar para a mesma de algoritmos menos eficientes como o *bubble sort*.

→ **HEAP SORT: USANDO ÁRVORES PARA ORDENAR LISTAS OU OUTRAS ESTRUTURAS**

O *heap sort* é um algoritmo de ordenação eficiente que utiliza uma estrutura de dados chamada *heap* (geralmente pronunciamos como “*ríp*”) para ordenar elementos. Em resumo, ele funciona assim:

1. **Construção do *heap*:** primeiramente, transformamos a lista ou *array* em um *heap* (geralmente um *heap* máximo). Um *heap* nada mais é do que uma árvore binária especial, em que cada nó pai é maior (nas árvores que chamamos de “*heap* máximo”) ou menor (nas árvores que chamamos de “*heap* mínimo”) que seus nós filhos. A construção do *heap* reorganiza os elementos de modo que essa propriedade seja satisfeita em toda a árvore.
2. **Ordenação:** depois de termos o *heap*, começamos o processo de ordenação. Removemos o elemento do topo do *heap* (que é o maior no *heap* máximo) e o colocamos na posição final da lista. Isso deixa um buraco no topo do *heap*, que preenchemos com o último elemento do *heap*, desfazendo temporariamente a propriedade de *heap*.
3. **Restauração do *Heap*:** restauramos a propriedade do *heap* fazendo um processo chamado “*heapify*”, que ajusta a árvore para que o maior elemento suba para o topo do *heap*. Isso é feito repetidamente até que todos os elementos tenham sido removidos do *heap* e colocados na lista.
4. **Resultado:** ao final do processo, todos os elementos são removidos do *heap* e colocados na lista em ordem crescente (se estamos usando um *heap* máximo).

Veja a animação a seguir para um exemplo. Veja que a lista é ordenada a partir da organização da árvore binária (*heap*). Isto é: usamos o *heap* para auxiliar o nosso trabalho de ordenação.



Fonte: https://commons.wikimedia.org/wiki/File:Heap_sort_example.gif

Como de costume, o que acha de pensarmos em uma analogia? Vamos usar a analogia de uma competição para explicar o funcionamento do *heap sort*. Imagine que você é o organizador de um *hackathon*. Se você não sabe, um *hackathon* é uma competição de equipes para desenvolver um software em um curto espaço de tempo para resolver um problema da vida real. Também é um ótimo jeito de fazer networking na área de TI e, em alguns casos, conseguir um estágio.

Agora, o seu desafio é o de encontrar **a melhor equipe** do *hackathon*. O problema é que são muitas equipes, e fica difícil comparar todas. Usando o *heap sort*, poderíamos pensar em algo assim:

1. **Construção do *heap***: imagine que todas as equipes do *hackathon* são inicialmente organizadas em uma estrutura que se assemelha a uma pirâmide, representando um *heap*. Nesta pirâmide, uma certa equipe (nó pai) tem uma solução considerada mais inovadora ou tecnicamente superior (tem uma pontuação mais alta) do que as equipes diretamente a seguir dela (nós filhos). Essa organização inicial é como construir um *heap* máximo.
2. **Ordenação**: a competição começa com a equipe no topo da pirâmide, que tem a solução considerada a mais inovadora. Essa equipe é então "removida" do *hackathon* e colocada na posição final da classificação, como a equipe vencedora. Afinal, a ideia da organização do *hackathon* é a de primeiramente publicar as piores equipes e, no final, as melhores.
3. **Restauração do *heap***: depois de remover a equipe do topo, o espaço vago é preenchido com a equipe que estava na base da pirâmide. Agora a estrutura pode ter perdido a propriedade do *heap* máximo, e então você reorganiza as equipes para que a próxima equipe mais inovadora suba até o topo da pirâmide. Este processo é como realizar o "*heapify*".
4. **Resultado**: você repete esse processo de remover a equipe do topo, reorganizar a pirâmide e preencher o espaço vago, até que todas as equipes sejam classificadas. No final, você terá uma lista de equipes ordenadas do projeto menos inovador para o mais inovador.

O *heap sort* é **particularmente eficaz para conjuntos de dados grandes**, pois sua performance é muito consistente: sua complexidade temporal no pior caso é $O(n \log n)$, o que o torna uma opção confiável para ordenação. Ele também tem a vantagem de ser um algoritmo *in-loco* (não requer espaço adicional significativo além do necessário para a lista original).

Casos de uso

E quando usamos esses algoritmos na realidade? Para começar, o ***merge sort*** é conhecido pela sua eficiência e estabilidade, tornando-o adequado para casos de uso como:

+ Ordenação de Listas Encadeadas

Devido à sua natureza de ordenação baseada em links, o *merge sort* é ideal para listas encadeadas, em que a sua implementação não requer memória extra para os ponteiros de elementos.

+ Ordenação externa

É frequentemente usado em situações de ordenação externa, como ordenar arquivos grandes que não cabem na memória, porque divide o problema em partes menores que são mais gerenciáveis.

+ Sistemas de banco de dados

Bancos de dados usam merge sort para operações de ordenação e mesclagem, especialmente quando envolvem grandes volumes de dados que precisam ser ordenados de forma eficiente.

Já o *quick sort* é amplamente utilizado e preferido para a ordenação geral devido à sua eficiência em média. Vejamos:

+ Linguagens de programação

Algumas implementações da função **sort()** em bibliotecas de linguagens de programação, como C++ (na STL) e Java (na API Collections), usam variações do Quick Sort chamadas de QuickSort3.

+ Bancos de dados

Também, usamos esta técnica para implementar algoritmos de ordenação em memória em sistemas de gerenciamento de banco de dados.

+ Sistemas de arquivos

É usado para ordenar itens como arquivos e pastas, aproveitando sua eficiência em listas grandes.

Finalmente, usamos o *heap sort* em casos em que precisamos de uma eficiência consistente e uma capacidade de ordenação *in-loco*. Ou seja, em casos como:

+ Ordenação de grandes conjuntos de dados

É útil para ordenar grandes conjuntos de dados, em que a eficiência consistente é mais importante do que a velocidade extrema em casos médios.

+ Implementações do algoritmo de *heap*

É frequentemente utilizado em algoritmos baseados em heap, como na criação de filas de prioridade, em que a ordenação e a manutenção da estrutura do heap são cruciais.

+ Sistemas de gerenciamento de banco de dados

Em situações em que a estabilidade não é uma preocupação, mas a previsibilidade do desempenho é importante, como em certas operações de banco de dados. Exemplos disso são em casos em que registros duplicados são raros ou em que a ordem original dos registros com o mesmo valor não é importante.

Métodos lineares de ordenação

Também temos os métodos lineares de ordenação. Estes métodos, também conhecidos como **algoritmos de ordenação não comparativos**, são um grupo de algoritmos de ordenação que não comparam os elementos entre si diretamente. Eles são chamados de "lineares" porque, em muitos casos, podem ordenar os elementos em tempo linear (ou seja, $O(n)$, em que n é o número de elementos a serem ordenados).

Isso os diferencia significativamente dos métodos de ordenação baseados em comparação, como os outros métodos que vimos até agora, que geralmente têm uma complexidade temporal de $O(n \log n)$ no melhor caso. Na prática, estes métodos lineares podem ser bem mais eficientes, mas somente em alguns casos de uso bem limitados. Também podem ser mais complexos para implementar e entender do que os outros métodos de ordenação que vimos até o momento. Em resumo, os métodos que veremos agora serão:

1. **Counting sort:** conta o número de ocorrências de cada elemento único e, em seguida, usa essa contagem para colocar cada elemento diretamente em sua posição correta.
2. **Radix sort:** ordena os elementos dígito a dígito, começando pelo dígito menos significativo até o mais significativo.
3. **Bucket sort:** divide os elementos em vários "baldes" e, em seguida, ordena esses baldes individualmente, geralmente usando um algoritmo de ordenação diferente.

Estes algoritmos são úteis em situações específicas, como, por exemplo, quando os elementos a serem ordenados têm um intervalo de valores limitado; para dados que podem ser facilmente categorizados em um número finito de categorias; ou quando os elementos são grandes, mas a chave pela qual eles são ordenados é pequena (como *strings* longas com um número limitado de caracteres únicos).

Vamos lá?

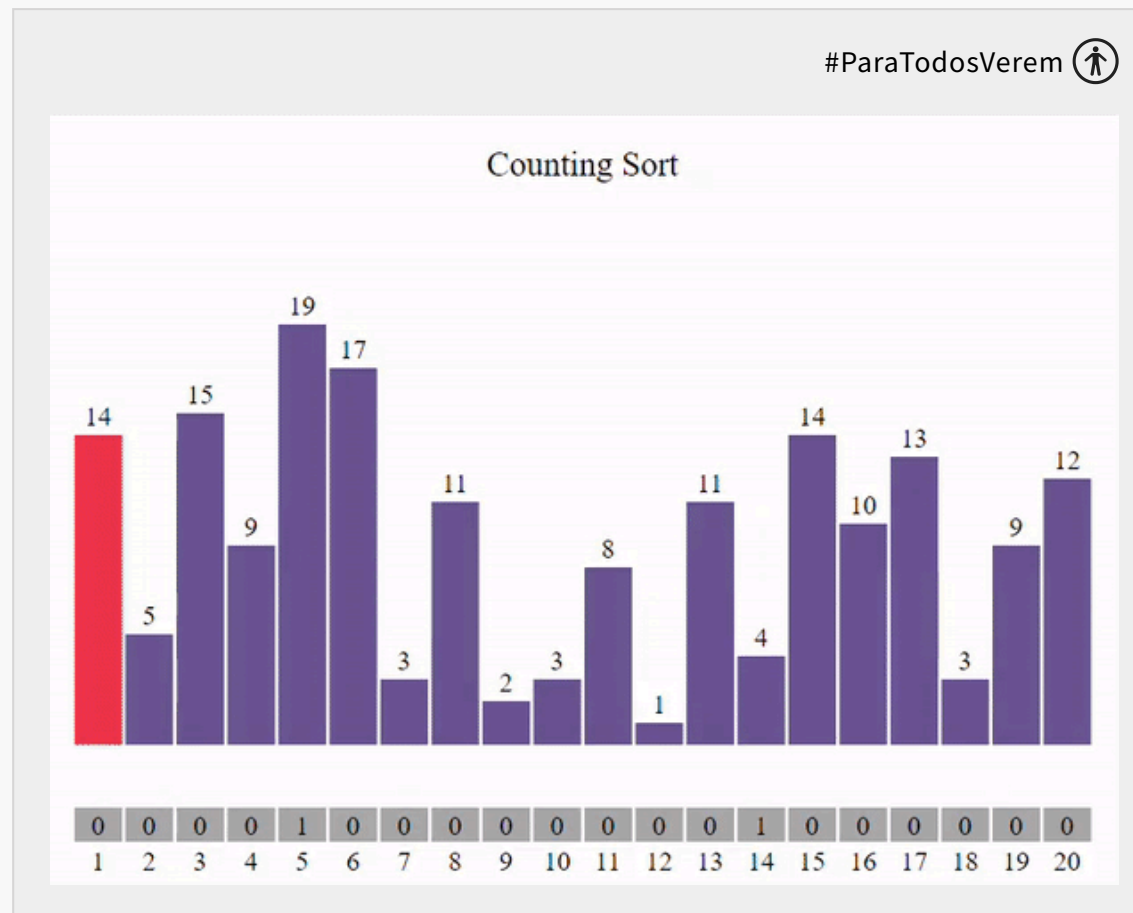
→ COUNTING SORT: AGRUPANDO VALORES REPETIDOS

O **counting sort** é um algoritmo de ordenação não-comparativo que conta quantas vezes um determinado valor se repetiu em um conjunto de dados e usa essas contagens para determinar a posição de cada elemento na sequência ordenada. Aqui está uma explicação simplificada do processo:

1. **Encontrando o intervalo de valores:** primeiramente, o algoritmo identifica o valor mínimo e máximo no conjunto de dados. Essa faixa de valores determina o tamanho do *array* de contagem.
2. **Inicializando o *array* de contagem:** um *array* de contagem é inicializado com um tamanho igual ao intervalo de valores dos elementos. Cada posição no *array* de contagem correspondente a um valor possível no conjunto de dados.
3. **Contando as ocorrências:** o algoritmo percorre o conjunto de dados e, para cada elemento, incrementa a contagem no índice correspondente do *array* de contagem. Assim, ao final desse passo, **cada posição do *array* de contagem contém o número de vezes que o valor correspondente aparece no conjunto original.**
4. **Acumular as contagens:** as contagens no *array* são então acumuladas sequencialmente. Isso significa que cada posição no *array* de contagem será ajustada para refletir a posição final do valor correspondente na sequência ordenada. Essencialmente, cada valor no *array* de contagem indica a posição final no *array* ordenado do último elemento com aquele valor.
5. **Construir a sequência ordenada:** finalmente, o algoritmo percorre o conjunto de dados novamente e usa o *array* de contagem para colocar cada elemento em sua posição correta no *array* ordenado. Conforme os elementos são colocados, as contagens no *array* de contagem são decrementadas.

Para entender isso melhor, imagine que você possui dezenas de lápis de cor. Algumas cores estão repetidas, e você precisa organizar os seus lápis. No *counting sort*, você precisaria primeiramente identificar todas as cores diferentes (intervalo de valores) e preparar potes ou compartimentos para cada cor (inicialização do *array* de contagem). Ao passar por cada lápis, você os coloca no pote correspondente à sua cor, contando assim a quantidade de lápis de cada cor (contagem de ocorrências). Depois, os potes são organizados na ordem desejada, refletindo a ordem final dos lápis. Finalmente, você esvazia os potes um por um, na ordem estabelecida, colocando os lápis de volta na caixa, o que resulta em uma caixa de lápis ordenados por cor.

Veja a animação a seguir para entender esta técnica. Observe o *array* embaixo da animação, em cinza: é ele que representa o *array* de contagem.



Fonte: GeeksForGeeks <https://www.geeksforgeeks.org/counting-sort-visualization-using-javascript/>

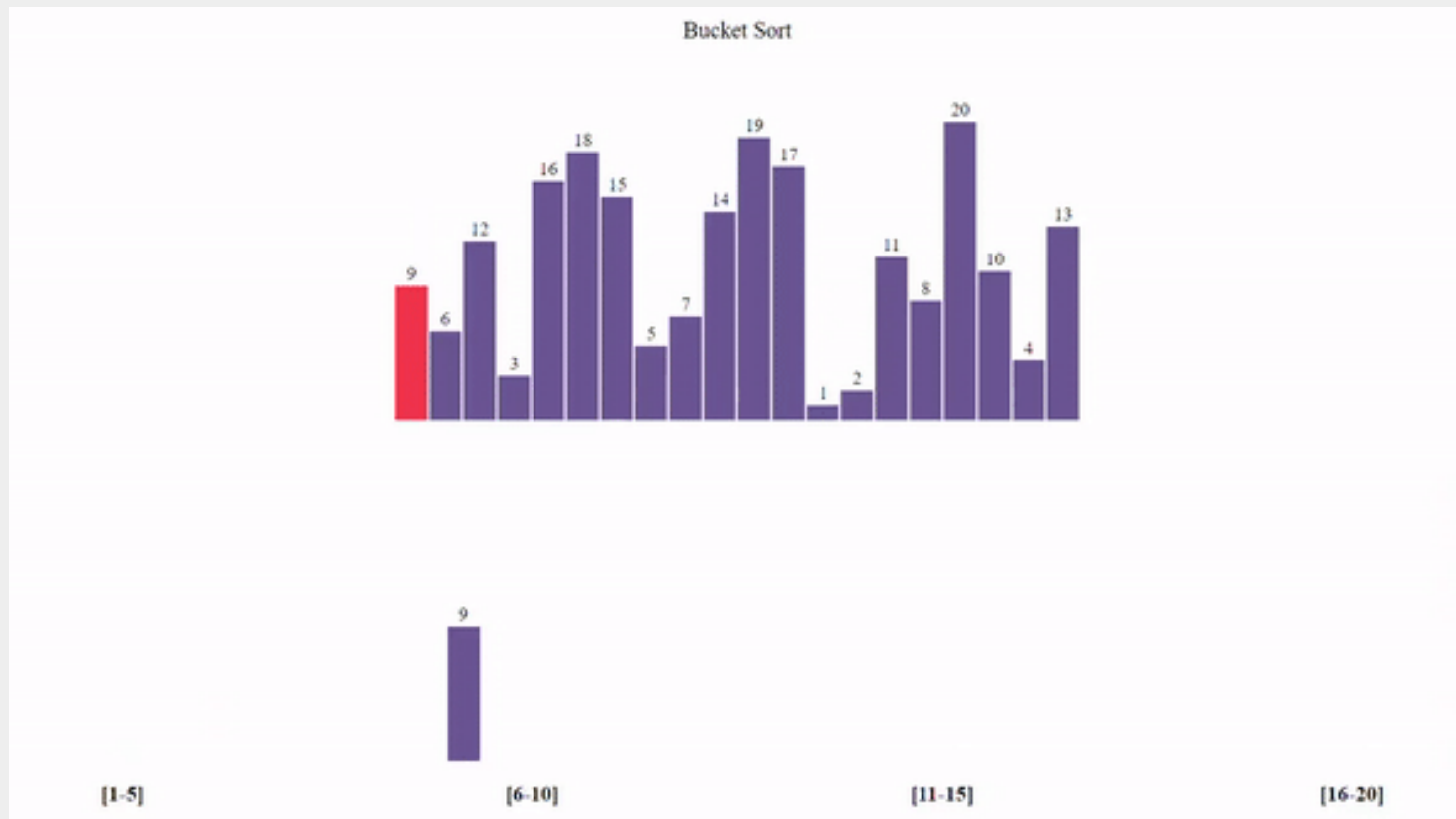
O *counting sort* é mais eficiente quando o intervalo de valores possíveis (diferença entre o valor máximo e mínimo) é não muito maior do que o número de elementos a serem ordenados. Ele também é particularmente eficaz para ordenar **grandes conjuntos de dados com um pequeno intervalo de valores distintos**. Como o *counting sort* não realiza comparações diretas entre os elementos do conjunto de dados, ele pode atingir uma complexidade de tempo linear, $O(n)$, em que n é o número de elementos a serem ordenados.

→ **BUCKET SORT: INTRODUZINDO OS BALDES**

O *bucket sort* é um algoritmo de ordenação que distribui elementos de um *array* em vários recipientes, ou "baldes" (*buckets*, em inglês), e depois ordena individualmente os elementos dentro de cada *bucket*. Após a ordenação, os elementos são reunidos para formar o *array* final ordenado. De forma resumida, o passo a passo deste algoritmo funciona assim:

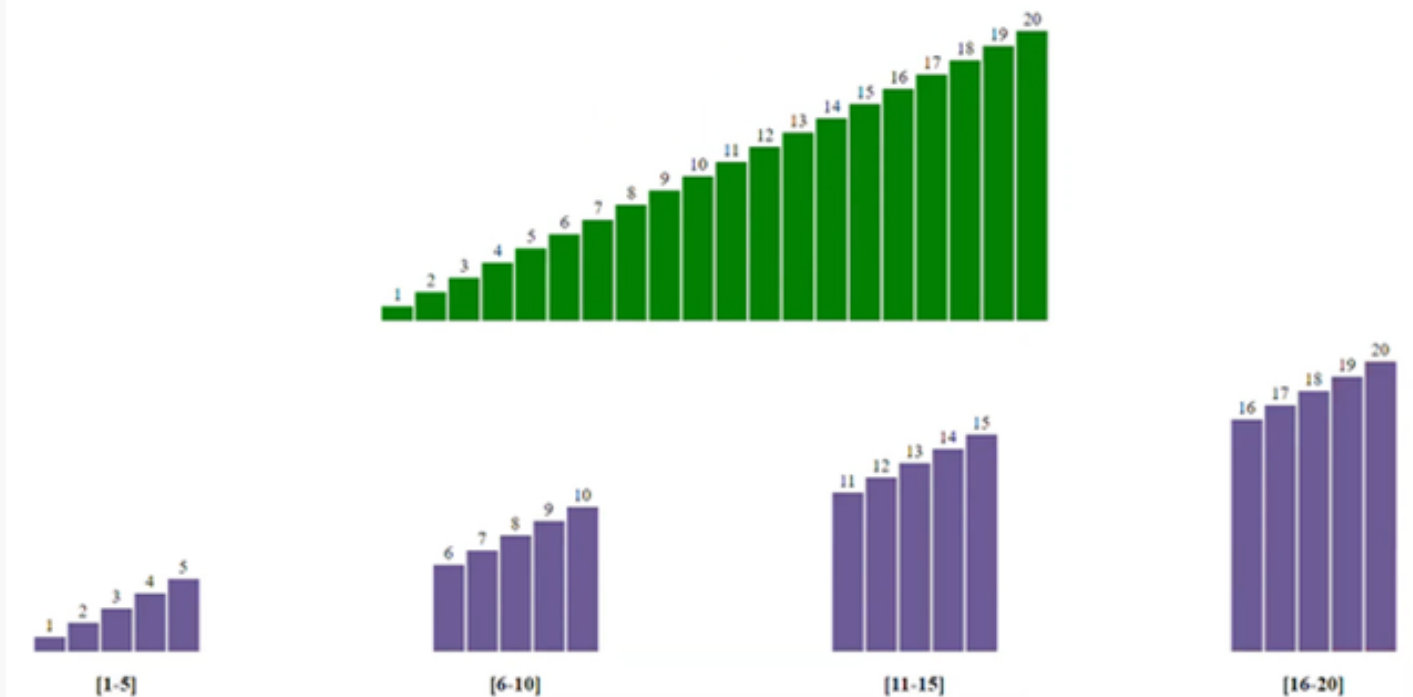
1. **Distribuição em *buckets*:** primeiramente, o algoritmo distribui os elementos do *array* original em vários *buckets*. Essa distribuição é geralmente feita com base em uma função que determina em qual *bucket* um elemento deve ser colocado. Por exemplo, em um cenário simples, essa função pode ser baseada no intervalo de valores dos elementos (como de 0 a 5, 6 a 10, 11 a 15 e assim sucessivamente).
2. **Ordenação individual dos *buckets*:** cada *bucket* é então ordenado individualmente. Isso pode ser feito usando qualquer algoritmo de ordenação, como o *insertion sort*, especialmente se os *buckets* contiverem poucos elementos.
 - Isso significa que o *bucket sort* pode ser combinado com outros métodos de ordenação neste passo em específico.
3. **Concatenação dos *buckets*:** após a ordenação dos *buckets*, os elementos de cada *bucket* são concatenados na ordem dos *buckets* para formar o *array* final ordenado.

Veja um exemplo mais visual na animação a seguir. Veja que temos quatro *buckets*: de 1 a 5, de 6 a 10, de 11 a 15, e de 16 a 20. Primeiramente, atribuímos os valores a cada *bucket*.



Fonte: GeeksForGeeks <https://www.geeksforgeeks.org/counting-sort-visualization-using-javascript/>

Depois disso, fica mais fácil de ordenarmos cada *bucket* e juntarmos os resultados em um vetor só, resultando em algo assim:



Fonte: GeeksForGeeks <https://www.geeksforgeeks.org/counting-sort-visualization-using-javascript/>

Para exemplificar este processo, imagine uma tarefa de organização por data de várias fotos que você tirou ao longo do ano. Para usar o *bucket sort*, primeiramente, você cria vários "*buckets*" (pastas), cada um representando um intervalo de tempo específico, como semanas ou meses. Você então percorre suas fotografias, colocando cada uma na pasta correspondente à semana ou mês em que foi tirada. Assim que todas as fotos estiverem distribuídas pelas pastas, você ordena as fotos dentro de cada pasta individualmente, talvez pelo dia.

Depois de organizar cada pasta você junta todas as fotos novamente em uma única coleção, começando com as fotos do *bucket* mais antigo e seguindo até o mais recente. Essa abordagem não só torna o processo de ordenação mais gerenciável, dividindo um grande problema em tarefas menores, mas também aproveita o conhecimento sobre a distribuição das fotos (por data) para tornar a ordenação mais eficiente.

O *bucket sort* é mais eficiente quando os elementos a serem ordenados são **distribuídos uniformemente nos buckets**. O desempenho do algoritmo pode sofrer se todos ou a maioria dos elementos forem distribuídos em um número limitado de *buckets*, pois isso desequilibra a carga entre eles. Em cenários ideais, em que os elementos são distribuídos uniformemente, o *bucket sort* pode alcançar a eficiência próxima a $O(n)$, especialmente se o número de elementos em cada *bucket* é pequeno. Este algoritmo é particularmente útil para ordenar dados com uma distribuição uniforme e quando se tem um conhecimento prévio sobre a distribuição dos elementos.

→ RADIX SORT: INDO UM DÍGITO DE CADA VEZ

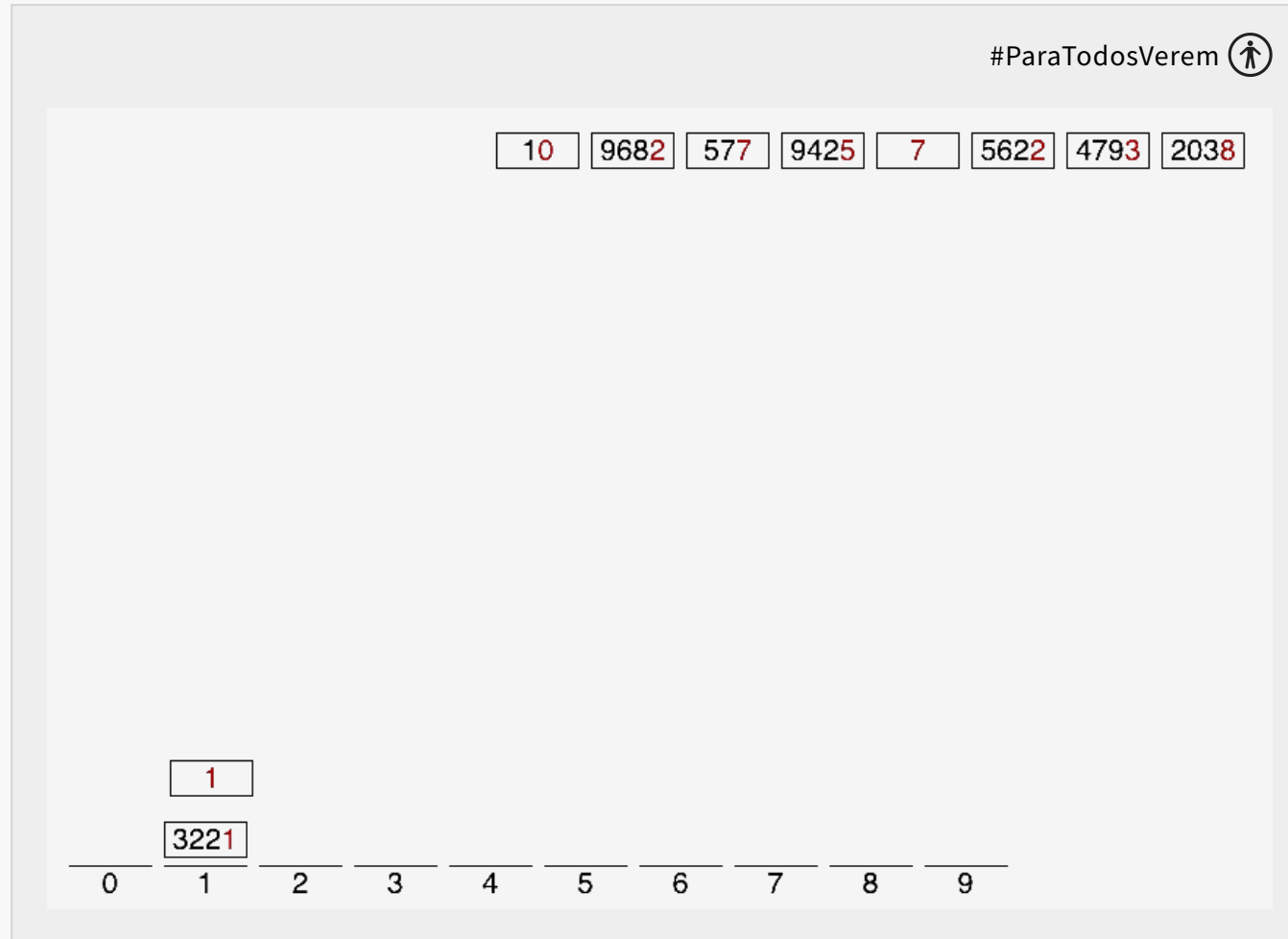
O *radix sort* é um algoritmo de ordenação não-comparativo que ordena os elementos processando suas chaves digitais ou **partes individuais**. Ao contrário dos algoritmos de ordenação baseados em comparações, o ***radix sort* organiza os dados com base em cada dígito ou caractere individual**, começando do menos significativo para o mais significativo (ou vice-versa, dependendo da implementação). Ele também reaproveita o conceito de *buckets* que vimos anteriormente. De forma resumida, ele funciona assim:

1. **Processamento por dígitos/caracteres:** o *radix sort* processa os elementos do conjunto de dados dígito por dígito. Em cada etapa, ele considera um dígito de cada elemento. Por exemplo, se estiver ordenando números, pode começar pelo dígito das unidades de cada número, depois o das dezenas, e assim por diante.
2. **Distribuição em *buckets*:** em cada iteração (ou passagem), os elementos são distribuídos em *buckets* com base no dígito que está sendo considerado. Para um sistema decimal, existiriam 10 *buckets* (de 0 a 9). Se estivéssemos falando sobre palavras, poderíamos ter um *bucket* para cada letra. Cada elemento é colocado no *bucket* correspondente ao seu dígito atual.
3. **Coleta dos *buckets*:** após distribuir todos os elementos nos *buckets*, eles são coletados de volta ao *array* original, mantendo a ordem dos *buckets*. Esta etapa é bem importante, uma vez que prepara os elementos para a próxima passagem, quando o próximo dígito mais significativo será considerado.
4. **Repetição para todos os dígitos:** o processo é repetido para cada dígito, começando pelo menos significativo e avançando para o mais significativo. A cada passagem, os elementos são distribuídos e coletados, e a ordem dos elementos se aproxima mais da ordem final desejada.

Sei que esse tipo de algoritmo é um pouco mais confuso de entender. Sendo assim, pense que você está organizando uma grande pilha de cartas de baralho em ordem, mas de uma maneira um pouco diferente. Em vez de olhar para o valor e o naipe da carta como um todo, você decide primeiramente organizar as cartas por naipe, independentemente de seu valor. Então, você cria quatro pilhas distintas: copas, espadas, ouros e paus.

Depois de separar as cartas por naipe, você começa a organizar cada pilha de naipe pelo número, começando pelo ás e indo até o rei. Este processo é muito semelhante ao *radix sort*, em que você primeiramente organiza as cartas (ou elementos) por uma característica menos significativa (naipe) e depois por uma característica mais significativa (valor numérico). Assim como no *radix sort*, em que cada dígito é ordenado individualmente começando pelo menos significativo, neste exemplo, você primeiramente separa por naipe e depois ordena cada grupo por valor. No final das contas, o seu baralho está ordenado corretamente.

Também para lhe ajudar, veja a animação a seguir. Perceba que a ordenação começa a cada dígito (destacado em vermelho escuro dentro de cada número). Depois, vamos ordenar pelo próximo dígito, e assim sucessivamente.



Fonte: National University of Singapore <https://visualgo.net/en/sorting>

O *radix sort* é mais eficiente quando o número de dígitos ou caracteres das chaves é conhecido e não é muito grande, e é particularmente útil para ordenar *strings*, números grandes ou outros tipos de dados em que as chaves podem ser decompostas em partes menores que podem ser ordenadas individualmente. Este algoritmo pode oferecer desempenho de tempo linear, $O(n \cdot k)$, em que n é o número de elementos e k é o número de dígitos ou caracteres nas chaves. **Ele é especialmente eficaz quando o tamanho do intervalo das chaves é maior do que o número de elementos a serem ordenados.**

Agora, e em quais casos da vida real usamos estes métodos? Primeiramente, o *counting sort* é particularmente útil quando o intervalo dos números é limitado. Isso significa, portanto, que ele é útil em casos como:

+ Ordenação de dados com intervalo limitado

É ideal para ordenar itens quando o intervalo de valores possíveis (como idades ou notas em uma escala) é limitado e conhecido.

+ Processamento de imagens

Pode ser usado no processamento de imagens para a ordenação de cores ou intensidades de *pixels*, em que o número de possíveis valores de *pixel* (como em imagens de 8 *bits*) é limitado.

+ Ordenação de caracteres em *strings*

Utilizado para ordenar caracteres individuais em *strings*, especialmente útil em algoritmos como a ordenação lexicográfica.

+ Algoritmos de criptografia

O *counting sort* pode ser usado para organizar chaves ou outros elementos de dados de acordo com sua frequência ou outros critérios específicos.

+ Classificação de dados categóricos

Para as pessoas que trabalham com IA, esta técnica pode ser ideal para classificar dados com um número limitado de categorias distintas, como classificar pessoas por faixa etária ou estudantes por notas.

Já o *radix sort* faz a ordenação por dígitos ou caracteres. Logo, pode ser bom em casos como:

+ Ordenação de *strings* longas

Radix sort é eficaz na ordenação de *strings* (como palavras ou nomes) em grandes volumes de dados, especialmente quando o comprimento das *strings* é limitado.

+ Sistemas de telefonia

Utilizado para ordenar números de telefone, que embora sejam longos, têm um número fixo de dígitos.

+ Ordenação de números grandes

Em cenários em que números muito grandes precisam ser ordenados, como números de identificação ou códigos de produtos.

+ Bancos de dados e indexação

Em sistemas de banco de dados, o *radix sort* pode ser usado para indexar e ordenar grandes volumes de dados com base em chaves numéricas ou *strings*.

Finalmente, o *bucket sort* divide o vetor em categorias e ordena-as individualmente. Isso significa que ele é usado em casos como:

+ Ordenação de dados distribuídos uniformemente

Bucket sort é útil quando os dados estão distribuídos uniformemente e podem ser divididos em *buckets* de forma eficiente, como em cenários de distribuição geográfica de dados.

+ Aplicações financeiras

Pode ser usado em análises financeiras em que os dados podem ser agrupados em intervalos de valores (como faixas de preço ou faixas de renda).

+ Ordenação de dados em redes de computadores

Em redes, para ordenar pacotes ou dados que têm uma distribuição relativamente uniforme sobre um intervalo de tempo.

Prós e contras

Agora, o que acha de compararmos todas estas técnicas? Entenda esta tabela comparativa como uma forma de entender melhor (de forma resumida) cada uma das principais técnicas de ordenação.

Algoritmo	Complexidade média	Complexidade pior caso	Estável	<i>In-place</i>	Notas
-----------	--------------------	------------------------	---------	-----------------	-------

<i>Bubble sort</i>	$O(n^2)$	$O(n^2)$	Não	Sim	Simples, mas ineficiente para grandes listas.
<i>Insertion sort</i>	$O(n^2)$	$O(n^2)$	Sim	Sim	Eficiente para pequenas listas ou quase ordenadas.
<i>Selection sort</i>	$O(n^2)$	$O(n^2)$	Não	Sim	Simples, mas ineficiente para grandes listas.
<i>Merge sort</i>	$O(n \log n)$	$O(n \log n)$	Sim	Não	Eficiente para grandes listas, mas requer espaço extra.
<i>Quick sort</i>	$O(n \log n)$	$O(n^2)$	Não	Sim	Eficiente em média, mas no pior caso é ruim.
<i>Heap sort</i>	$O(n \log n)$	$O(n \log n)$	Não	Sim	Bom para dados grandes, mas não é estável.
<i>Counting sort</i>	$O(n + k)$	$O(n + k)$	Sim	Não	Rápido para intervalo limitado de valores.
<i>Bucket sort</i>	$O(n + k)$	$O(n^2)$	Sim	Não	Bom para dados uniformemente distribuídos.
<i>Radix sort</i>	$O(n * k)$	$O(n * k)$	Sim	Não	Eficiente para números e <i>strings</i> longos.

| Conclusão

Nesta unidade, exploramos uma variedade de técnicas de ordenação, cada uma com suas peculiaridades, vantagens e desvantagens, adequadas para diferentes cenários e requisitos. É claro que existem mais técnicas do que somente as nove que vimos aqui, mas essas estão entre as mais comuns. Algoritmos como *bubble sort*, *insertion sort* e *selection sort*, apesar de sua simplicidade e fácil implementação, mostram limitações em termos de eficiência, especialmente com grandes conjuntos de dados, devido à sua complexidade $O(n^2)$.

Por outro lado, algoritmos mais avançados como *merge sort*, *quick sort* e *heap sort* oferecem uma eficiência significativamente maior, com complexidades de tempo médio e no pior caso de $O(n \log n)$, tornando-os escolhas preferenciais para conjuntos de dados maiores. Esses métodos utilizam estratégias mais sofisticadas, como dividir para conquistar e estruturas de dados em *heap*, para melhorar o desempenho da ordenação.

Além disso, técnicas não comparativas como *counting sort*, *bucket sort* e *radix sort* apresentam uma abordagem diferente, evitando comparações diretas e focando na manipulação de elementos com base em suas características individuais, como dígitos ou chaves. Estes algoritmos são particularmente eficazes em cenários específicos, como quando se lida com um intervalo limitado de valores ou dados uniformemente distribuídos, e podem atingir uma eficiência próxima de $O(n)$ em condições ideais.

Como comentamos algumas vezes até agora, vale reforçar que a escolha do algoritmo de ordenação correto depende da natureza dos dados, dos requisitos de desempenho e de espaço, e da necessidade de estabilidade na ordenação.

| Referências

TENENBAUM, A. M. **Estrutura de dados usando C**. São Paulo: Makron Books, 1995.

